Medical Diagnostic Retrieval System

Final Project Report

Team: Arunya & Srijan Reddy Sankepally

Course: Z2004 — Database Management Systems — Even Semester 2026

Track: A — RAG Pipeline **Engine:** PostgreSQL 16 + pgvector

GitHub: https://github.com/Arunyaiitm/medical-diagnostic-retrieval-system

---

1. Introduction

The Medical Diagnostic Retrieval System is a RAG-powered (Retrieval-Augmented Generation) query system for hospital radiology records. It allows doctors to search patient diagnostic data using natural language instead of writing SQL queries. The system combines structured SQL filtering with semantic similarity search over clinical notes using PostgreSQL's pgvector extension.

For example, a doctor can type "Show me pneumonia cases in male patients over 50" and the system returns matching patient records with diagnoses, prescriptions, doctor information, and relevance scores — all grounded in the actual database content.

The project is primarily DBMS-focused: the core work involves schema design, normalization, indexing, query optimization, and stored procedures. The RAG layer is a thin Python wrapper that makes the database accessible through natural language.

2. Data Source

The primary dataset comes from the NIH Chest X-Ray Dataset (Sample), publicly available on Kaggle. It contains 5,606 de-identified chest X-ray metadata records with patient demographics (ID, age, gender), 14 thoracic disease labels, follow-up numbers, view positions, and image dimensions. The dataset is public domain, released by the NIH Clinical Center.

Supporting relational data is generated by a documented Python seed script (`seed_data.py`) that creates 5 departments, 30 doctors, visit records, and prescription records around the real NIH data. The seed script enforces realistic constraints: doctor specializations are mapped to diagnosis types (e.g., Pulmonologists handle Pneumonia, Cardiologists handle Cardiomegaly), visit dates are derived from follow-up numbers, and prescriptions use real medication-to-condition mappings (e.g., Pneumonia → Azithromycin 500mg, Effusion → Furosemide 40mg).

3. Database Schema

The database consists of 7 normalized tables, all in Third Normal Form (3NF). The total dataset contains 26,389 rows across all tables.

Table	Rows	PK	Foreign Keys	Source
departments	5	department_id	—	Seed
doctors	30	doctor_id	department_id → departments	Seed
patients	4,230	patient_id	—	NIH
visits	5,606	visit_id	patient_id → patients, doctor_id → doctors	NIH+Seed
scans	5,606	scan_id	visit_id → visits	NIH
diagnoses	6,978	diagnosis_id	scan_id → scans, doctor_id → doctors	NIH
prescriptions	3,934	prescription_id	diagnosis_id → diagnoses	Seed

3.1. ER Diagram

The ER diagram follows Chen notation with rectangles for entities, ovals for attributes (dashed for derived, double-bordered for composite), and diamonds for relationships. All relationships are 1:M with appropriate partial and total participation constraints. The full ER diagram is included in `schema/er_diagram.pdf` (2 pages: diagram + 3NF argument with functional dependencies).

3.2. Normalization (3NF)

All 7 tables satisfy Third Normal Form. Each table has a single primary key, all attributes are atomic (1NF), every non-key attribute is fully dependent on the primary key (2NF), and there are no transitive dependencies (3NF). Foreign keys like `department_id` in doctors are references to other tables, not transitive dependencies.

3.3. Constraints

The schema enforces data integrity through CHECK constraints (gender M/F, age 0–130, severity MILD/MODERATE/SEVERE/CRITICAL, 14 NIH disease labels), UNIQUE constraints (department_name, email, image_filename), and foreign key constraints with ON DELETE CASCADE/RESTRICT policies.

3.4. pgvector Integration

The `diagnoses` table includes a `notes_embedding VECTOR(384)` column that stores 384-dimensional embeddings generated by the sentence-transformers model (all-MiniLM-L6-v2). This enables cosine similarity search over clinical notes using pgvector's `<=>` operator. All 6,981 diagnosis records have embeddings populated.

4. SQL Queries

Ten SQL queries demonstrate coverage of all required query types. Each query represents a realistic hospital operation.

#	Type	Description
1	Aggregation	Count of diagnoses per disease type (GROUP BY + ORDER BY)
2	Aggregation	Average patient age per department (AVG across 4 joins)
3	Join (4 tables)	Full diagnostic history: patient → visit → scan → diagnosis → prescription
4	Join (3 tables)	Doctors and diagnosis counts grouped by department
5	Subquery	Patients with more diagnoses than average (nested subquery)
6	Subquery	Scans with no diagnosis yet (NOT IN subquery)
7	CTE	Patient diagnostic journey timeline (WITH + HAVING)
8	CTE	Monthly diagnosis trends per disease (DATE_TRUNC)
9	Window (RANK)	Rank doctors by diagnoses within each department
10	Window (ROW_NUMBER)	Running total of visits per patient over time

All 10 queries run on a fresh database with zero errors and return correct, meaningful results. Coverage includes 2 aggregations, 2 joins, 2 subqueries, 2 CTEs, and 2 window functions.

5. Performance Optimization

5.1. Index Design

We created 11 B-tree indexes targeting columns used in joins, WHERE clauses, and GROUP BY operations.

Index	Column(s)	Justification
idx_patients_age	patients(age)	Range filter: WHERE age > 50
idx_patients_gender	patients(gender)	Equality filter: WHERE gender = 'M'
idx_visits_patient_id	visits(patient_id)	FK join: patients → visits
idx_visits_doctor_id	visits(doctor_id)	FK join: doctors → visits
idx_visits_date	visits(visit_date)	Date range queries
idx_scans_visit_id	scans(visit_id)	FK join: visits → scans
idx_diagnoses_scan_id	diagnoses(scan_id)	FK join: scans → diagnoses
idx_diagnoses_finding	diagnoses(finding_label)	Disease type filter
idx_diagnoses_doctor_id	diagnoses(doctor_id)	FK join: doctors → diagnoses
idx_prescriptions_diagnosis_id	prescriptions(diagnosis_id)	FK join: diagnoses → prescriptions
idx_diagnoses_finding_date	diagnoses(finding_label, diagnosis_date)	Composite: disease + date range

5.2. Performance Results

Three representative queries were benchmarked before and after indexing using EXPLAIN ANALYZE under identical conditions (same dataset, same machine, second run to avoid cold-cache effects).

Query	Before (ms)	After (ms)	Speedup	Key Plan Change
Q1: Pneumonia lookup	1.867	0.820	2.3×	Seq Scan → Bitmap Index Scan
Q2: Doctor ranking	5.929	2.891	2.1×	Planning time 0.933 → 0.097 ms
Q3: Monthly trends	7.736	4.012	1.9×	Faster hash joins via FK indexes

For Query 1, the composite index on `(finding_label, diagnosis_date)` allowed PostgreSQL to use a Bitmap Index Scan to locate 64 Pneumonia rows across 50 heap blocks, instead of scanning all 6,978 diagnosis rows. For Query 3, PostgreSQL kept a Sequential Scan for the date filter (43% selectivity makes index scan inefficient) but FK indexes accelerated hash join construction.

Full EXPLAIN ANALYZE output for all three queries is included in `queries/explain_output.txt`.

6. Stored Procedure

The `auto_prescribe` stored procedure simulates a clinical decision support system. Given a `diagnosis_id`, it looks up the finding label, maps it to the appropriate medication using real drug-to-condition mappings, and inserts a prescription record into the prescriptions table.

```
-- Usage
SELECT auto_prescribe(6979);
-- Result: Prescribed Azithromycin 500mg (Once daily, 5 days)
--         for Pneumonia (diagnosis_id: 6979)

-- Verify
SELECT * FROM prescriptions WHERE diagnosis_id = 6979;
-- Returns: Azithromycin, 500mg, Once daily, 5 days
```

The procedure handles all 14 disease types from the NIH dataset. For "No Finding" and "Nodule", it returns a message instead of inserting a prescription. The function modifies data by inserting into the prescriptions table, satisfying the requirement for a stored procedure that modifies data.

7. RAG Application

7.1. Architecture

The RAG pipeline follows this flow:

1. Doctor enters a natural language query (e.g., "severe pneumonia in elderly patients")
2. The system parses the query for SQL filters: disease type, gender, age range, severity
3. Simultaneously, the query is embedded into a 384-dimensional vector using sentence-transformers (all-MiniLM-L6-v2)
4. A hybrid SQL query combines structured filtering (WHERE clauses) with pgvector cosine similarity search (ORDER BY embedding distance)
5. Results are returned with patient details, diagnosis, prescription, doctor info, and a relevance score

7.2. Implementation

The application is implemented in Python with two interfaces:

Command-line interface (`app/main.py`): A terminal-based REPL where doctors type queries and see results formatted as a report. Supports all filter types and falls back to SQL-only search if embeddings are unavailable.

Web interface (`app/app.py`): A Streamlit web application with a search bar, sidebar showing database statistics and disease distribution, an auto-prescribe feature, and styled result cards with relevance scores. The web interface provides a professional demo-ready experience.

7.3. Query Parsing

The parser extracts structured filters from natural language using pattern matching:

```
"pneumonia in male patients over 50"
  -> finding_label ILIKE '%pneumonia%'
  -> gender = 'M'
  -> age > 50

"severe effusion cases"
  -> finding_label ILIKE '%effusion%'
  -> severity = 'SEVERE'

"female patients between 20 and 30 with infiltration"
  -> gender = 'F'
  -> age BETWEEN 20 AND 30
  -> finding_label ILIKE '%infiltration%'
```

7.4. Semantic Search

The pgvector cosine similarity search complements SQL filtering by ranking results by semantic relevance to the query. The embedding model (all-MiniLM-L6-v2) produces 384-dimensional vectors. The similarity score is computed as `1 - (embedding <=> query_vector)`, where `<=>` is pgvector's cosine distance operator. Results are ordered by similarity, so the most semantically relevant records appear first.

7.5. Sample Output

```
> Show me pneumonia cases in male patients over 50
```

```
Found 10 matching records

[1] Patient #23089 - Male, Age 59
    Diagnosis: Pneumonia (MODERATE)
    Doctor: Dr. Ahmed Saleh | Pulmonology
    Prescription: Amoxicillin 500mg
    Relevance: 65.8%

[2] Patient #13952 - Male, Age 59
    Diagnosis: Pneumonia (MODERATE)
    Doctor: Dr. Mohamed Juma | Pulmonology
    Prescription: Amoxicillin 500mg
    Relevance: 65.8%
```

8. Repository Structure

The project follows the required folder structure for final submission:

```
medical-diagnostic-retrieval-system/
  schema/          -> ER diagram + schema.sql (7 tables, 3NF)
  data/            -> sample_labels.csv + seed_data.py
  queries/         -> queries.sql + performance.sql + explain_output.txt
  app/             -> main.py + app.py + generate_embeddings.py
  report/          -> This report
  demo/            -> 5-minute demo video
  .env.example     -> Config template (no secrets)
  .gitignore       -> Excludes .env, .DS_Store, __pycache__
  requirements.txt -> Python dependencies
  README.md        -> Setup instructions + project overview
```

No secrets (API keys, passwords) are committed. The `.env` file is excluded via `.gitignore`, and `.env.example` provides a template.

9. Tech Stack

Component	Technology
Database	PostgreSQL 16.13
Vector search	pgvector 0.8.2
Embedding model	sentence-transformers (all-MiniLM-L6-v2, 384 dims)
Backend	Python 3.13, psycopg2
Web frontend	Streamlit
Environment	macOS (Apple Silicon), .env for secrets
Version control	Git + GitHub

10. Innovation

Several features go beyond the minimum project requirements:

Streamlit web interface: A professional, browser-based UI that provides a demo-ready experience with styled result cards, sidebar statistics, and interactive controls. This goes beyond the required command-line interface.

Hybrid search: The RAG pipeline combines SQL filtering with pgvector semantic search, providing both precision (exact matches on disease, gender, age) and recall (semantically similar clinical notes ranked by relevance).

Real medication mappings: The `auto_prescribe` stored procedure uses evidence-based medication-to-condition mappings rather than random data, making the system clinically realistic.

Composite index: The `(finding_label, diagnosis_date)` composite index demonstrates ad-

vanced indexing strategy, allowing PostgreSQL to satisfy both disease type and date range filters in a single index lookup.

Natural language parsing: The query parser extracts structured filters from free-text input, supporting disease names, gender, age ranges (over/under/between), and severity levels without requiring the user to know SQL.

11. Limitations

The system has several limitations that are important to acknowledge. The natural language parser uses pattern matching rather than a full NLP model, so queries must contain recognizable keywords (e.g., "pneumonia", "male", "over 50"). Queries like "patients with breathing problems" would not match "Pneumonia" because the parser looks for exact disease names.

The clinical notes in the `diagnoses` table are template-generated rather than real clinical text, which limits the diversity of semantic search results. In a production system, real doctor-written notes would produce more varied and useful embeddings.

The dataset is a sample (5,606 records) from the full NIH Chest X-Ray dataset (112,120 records). Performance characteristics may differ at production scale.

The system does not use an LLM to generate natural language responses. It returns structured records ranked by relevance. Adding an LLM response generation layer would improve the user experience but was outside the scope of this DBMS-focused project.

12. Future Work

Several enhancements could improve the system. Integrating an LLM (e.g., GPT or Claude) to generate natural language summaries from the retrieved records would make responses more conversational. Adding an HNSW index on the `notes_embedding` column would accelerate similarity search at larger dataset scales. Supporting more complex natural language queries through a proper NLP parser or LLM-based query understanding would handle queries like "patients who got worse after treatment." Expanding the dataset to the full 112,120 NIH records would provide a more realistic test of performance at scale. Finally, adding user authentication and role-based access control would make the system suitable for real hospital deployment.

13. Conclusions

The Medical Diagnostic Retrieval System demonstrates how a well-designed PostgreSQL database can power a natural language query interface for medical records. The project is primarily DBMS-focused: 7 normalized tables in 3NF with 26,389 rows, 11 optimized indexes providing 1.9×–2.3× speedup, 10 SQL queries covering all required types, a stored procedure that automates prescription generation, and pgvector integration for semantic search over 6,981 embedded clinical notes.

The RAG layer adds a practical interface that makes the database accessible to doctors who do not know SQL, while keeping all data grounded in the relational schema. The Streamlit web interface provides a professional, demo-ready experience that showcases the full pipeline from natural language input to database-backed results.

14. AI Usage Disclosure

Used Claude (Anthropic) for brainstorming schema design, debugging SQL queries, drafting the seed script logic, and building the RAG application. All output was reviewed, understood, and adapted before use. No large blocks of generated code were used without reading, understanding, and modification.